Detecção de vulnerabilidades com Qwen2.5-Coder no PrimeVul: desempenho global, pares vulnerável/corrigida e implantação

Autoria a confirmar

[1]Programa de Pós-Graduação em Ciência da Computação
Universidade Federal do Maranhão

Abstract. *Global metrics, used to assess vulnerability detectors based on language models, do not show whether a model separates a vulnerable function from its fixed version. This work examines that relation for 3B and 7B Qwen2.5-Coder configurations on PrimeVul: prompted instruction-tuned variants, classifiers fine-tuned with LoRA on a quantized base, and a heuristic that uses only function length. The configurations were evaluated on the global test set and on 564 vulnerable/fixed pairs. The heuristic reached a global AUC of 0.810, but its pairwise ranking score, which gives half credit to ties, was only 21.10%. The classifier with the highest global F1 (0.262) reached a pairwise ranking score of 37.50%. For the 7B model, mean pairwise ranking stayed close to 50%, and the NF4 deployment used less memory and more time per function than BF16. These exploratory results indicate that, under these conditions, global performance should be read together with paired evaluation and deployment conditions.*

Resumo. *Métricas globais, usadas para avaliar detectores de vulnerabilidades baseados em modelos de linguagem, não indicam se o modelo distingue uma função vulnerável de sua versão corrigida. Este trabalho examina essa relação em configurações Qwen2.5-Coder de 3B e 7B no PrimeVul: variantes instruídas por prompt, classificadores ajustados com LoRA sobre base quantizada e uma heurística que usa apenas o comprimento da função. As configurações foram avaliadas no teste global e em 564 pares vulnerável/corrigida. A heurística obteve AUC global de 0,810, mas sua ordenação pareada, métrica que dá meio crédito aos empates, foi de apenas 21,10%. O classificador de maior F1 global (0,262) teve ordenação pareada de 37,50%. No 7B, a ordenação média ficou próxima de 50%, e a implantação NF4 usou menos memória e mais tempo por função que a BF16. Esses resultados exploratórios indicam que, nessas condições, o desempenho global deve ser interpretado junto à avaliação pareada e às condições de implantação.*

1. Introdução

A detecção de vulnerabilidades em código-fonte pode ser formulada como classificação de funções: dado o corpo de uma função C/C++, um detector indica se ela contém uma vulnerabilidade. Essa formulação tem uma finalidade delimitada, que é indicar a analistas de segurança quais funções examinar, embora a localização das linhas defeituosas continue exigindo inspeção manual [Fu e Tantithamthavorn 2022]. Os detectores aprendidos para essa tarefa passaram de representações do código em grafos [Zhou et al. 2019]

a Transformers pré-treinados em código [Fu e Tantithamthavorn 2022] e, mais recentemente, a modelos de linguagem de grande porte (LLMs, do inglês *large language models*) especializados em código, como a família Qwen2.5-Coder [Hui et al. 2024]. Essa trajetória, detalhada na Seção 2, ampliou os modelos disponíveis, mas a utilidade de cada detector para aquela finalidade depende de como seu desempenho é medido.

Estudos de avaliação mostraram que dados e protocolos podem favorecer resultados que não se mantêm em condições realistas. Chakraborty et al. [2020] observaram queda acentuada no desempenho de detectores profundos avaliados em um cenário realista e a associaram à duplicação de dados, a uma distribuição irreal das classes e a modelos que frequentemente não aprendem características relacionadas à vulnerabilidade. Ding et al. [2024] encontraram cópias exatas e vulnerabilidades clonadas repartidas entre treino e teste de conjuntos anteriores e propuseram o PrimeVul; nesse benchmark, um modelo de 7B que alcançava F1 elevado no BigVul obteve F1 muito baixo. Por outro caminho, Risse e Böhme [2024] verificaram que modelos treinados em um conjunto típico não distinguiram funções vulneráveis de suas versões corrigidas, com acurácia média inferior à de escolhas aleatórias.

Esses achados mostram que uma métrica global pode ser alta por razões alheias à vulnerabilidade. Quando a classe vulnerável é rara, como no teste do PrimeVul, em que representa 2,68% das funções, a curva ROC pode ser enganosa quanto à confiabilidade de um classificador [Saito e Rehmsmeier 2015]. Além disso, qualquer característica correlacionada ao rótulo, como o tamanho da função, pode gerar discriminação global sem relação com o defeito. A comparação entre a função vulnerável e sua versão corrigida reduz essa possibilidade, porque as duas versões compartilham projeto e contexto e diferem pela correção. Por esse motivo, o PrimeVul acrescenta às métricas globais uma avaliação pareada e uma medida da taxa de falsos negativos sob restrição de falsos positivos [Ding et al. 2024]. Com um protocolo próprio, Huang et al. [2026] mostraram que LLMs ajustados sem raciocínio explícito obtiveram escores altos quando a função vulnerável era contrastada com código normal não relacionado, mas falharam diante das versões corrigidas.

Em um uso prático, a qualidade também precisa ser lida junto aos recursos exigidos. A adaptação de baixo posto (LoRA) ajusta matrizes adicionais e mantém congelados os pesos pré-treinados [Hu et al. 2021]. QLoRA estende essa ideia ao manter a base quantizada em quatro bits durante o ajuste [Dettmers et al. 2023]. Com isso, um mesmo ajuste pode ser implantado de formas que diferem em memória e em tempo de processamento, e a escolha entre elas depende de saber se a discriminação se mantém. Nesse contexto, este trabalho analisa a relação entre desempenho global, discriminação pareada e recursos de implantação em configurações Qwen2.5-Coder de 3B e 7B avaliadas no PrimeVul. A primeira questão é se o desempenho global dessas configurações acompanha sua capacidade de atribuir à função vulnerável um escore maior que o de sua versão corrigida, tendo como referência uma heurística baseada somente no comprimento da função. A segunda é como duas implantações completas do mesmo classificador 7B se comparam em discriminação pareada, memória e tempo: uma em NF4 com adaptador separado e outra em BF16 com adaptador mesclado. Para as configurações executadas, o estudo mostra que discriminação global elevada coexistiu com ordenação pareada desfavorável, descreve a dispersão entre três sementes de treinamento do 7B e quantifica a troca entre memória e

tempo nas duas implantações. A análise é retrospectiva e exploratória, pois o conjunto de teste foi consultado durante o desenvolvimento das receitas.

O restante do artigo está organizado da seguinte forma. A Seção 2 apresenta os fundamentos e os trabalhos relacionados. A Seção 3 descreve dados, configurações, regras de decisão e análise estatística. A Seção 4 apresenta os resultados, discutidos na Seção 5 à luz da literatura. A Seção 6 reúne as limitações, e a Seção 7 conclui o trabalho.

2. Fundamentação e trabalhos relacionados

2.1. Da detecção aprendida aos modelos de linguagem de código

Os detectores aprendidos para funções C/C++ diferem sobretudo na representação do código. No Devign, a árvore sintática serve de base para codificar, em um único grafo heterogêneo, as dependências de controle e de dados em diferentes níveis, e a classificação ocorre no nível do grafo [Zhou et al. 2019]. Os autores avaliaram essa representação em dados rotulados manualmente de quatro bibliotecas C, o que vincula os resultados a poucos projetos. O LineVul trocou a representação explícita em grafos por um Transformer pré-treinado em código, avaliado em mais de 188 mil funções C/C++, e acrescentou à predição por função a localização de linhas por meio da atenção [Fu e Tantithamthavorn 2022]. O presente estudo mantém a formulação por função e não avalia a localização de linhas.

Com os LLMs de código, a tarefa pode ser executada de duas maneiras. Na primeira, uma instrução textual solicita uma resposta, e um analisador converte o texto em rótulo; esse uso depende de variantes ajustadas para seguir instruções e não produz, em geral, um escore contínuo. Na segunda, uma camada de classificação é acoplada ao modelo e produz um logit por classe; a diferença entre eles fornece um escore que pode ser ordenado e comparado a limiares. A família Qwen2.5-Coder, oferecida em seis tamanhos, de 0,5B a 32B parâmetros, e pré-treinada em mais de 5,5 trilhões de tokens [Hui et al. 2024], permite examinar as duas formas de uso em escalas próximas. Nos experimentos de Chen et al. [2023], LLMs superaram modelos baseados em grafos e foram apontados como direção promissora para a detecção. A especialização em código, no entanto, não basta para garantir a detecção. Ullah et al. [2024] avaliaram oito LLMs em 228 cenários de código, ao longo de oito dimensões, e encontraram respostas não determinísticas, raciocínio incorreto ou infiel e baixa robustez a alterações nas condições de avaliação. A família Qwen2.5-Coder não integrou esse grupo, e os resultados desse trabalho não se transferem ao protocolo local.

2.2. Qualidade dos dados e protocolos de avaliação

Parte da literatura desloca a atenção dos modelos para os dados. Além de apontar a duplicação e a distribuição irreal das classes, Chakraborty et al. [2020] relataram que balancear os dados de treino entre classes melhorava a predição no cenário realista que estudaram. O DiverseVul reuniu 18.945 funções vulneráveis, de 150 categorias CWE, e 330.492 funções não vulneráveis, extraídas de 7.514 commits [Chen et al. 2023]. Seus autores também indicaram a dificuldade de generalizar para projetos não vistos no treino e quantificaram, por inspeção amostral, ruído considerável nos rótulos de vulnerabilidade, tanto em seu conjunto quanto em conjuntos anteriores. Esses resultados mostram que o

rótulo associado a uma função derivada de um commit de correção nem sempre corresponde à vulnerabilidade; as taxas medidas por esses autores, porém, não se aplicam ao PrimeVul.

No PrimeVul, a rotulação mais rigorosa busca maior exatidão dos rótulos, enquanto a deduplicação e a divisão cronológica reduzem o risco de vazamento entre treino e teste [Ding et al. 2024]. Na avaliação, o benchmark propõe o VD-Score, que mede a taxa de falsos negativos depois de ajustar o detector para manter os falsos positivos abaixo de um limite fixo, como 0,5%, e uma avaliação pareada da capacidade de distinguir a amostra vulnerável de sua contraparte corrigida. O VulnPatchPairs, de Risse e Böhme [2024], adota um desenho semelhante, ao contrapor funções vulneráveis a suas versões corrigidas. Esses autores também treinaram modelos com dados aumentados por transformações que preservam a semântica e os avaliaram com transformações diferentes, mostrando que os modelos se ajustavam a características sem relação com o rótulo introduzidas pelas transformações. O TrapEval, de Huang et al. [2026], estendeu essa investigação a LLMs ajustados. O protocolo contrasta os conjuntos V2P, que pareia código vulnerável com sua versão corrigida, e V2N, que o pareia com código normal não relacionado. Ele combina ainda testes entre conjuntos, perturbações que preservam a semântica e uma análise da distância semântica, em cinco LLMs ajustados com e sem raciocínio explícito.

A escolha das métricas também condiciona o que se observa. Em dados desbalanceados, a linha de base da curva ROC é fixa, enquanto a da curva precisão-revocação depende da razão entre positivos e negativos; por isso, a ROC pode ser enganosa quanto à confiabilidade do classificador [Saito e Rehmsmeier 2015]. Esse argumento não implica que a AUC aumente automaticamente com o desbalanceamento, mas justifica reportá-la junto à precisão e a pontos de operação com baixa taxa de falsos positivos.

2.3. Adaptação eficiente e implantação

Na LoRA, a atualização de uma matriz pré-treinada $W_0 \in \mathbb{R}^{d\times k}$ é representada pelo produto de duas matrizes de baixo posto, de modo que o peso adaptado é $W_0 + \Delta W = W_0 + BA$, com $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$ e $r \ll \min(d, k)$. Durante o treino, W_0 permanece congelada e apenas A e B recebem atualizações; o desenho linear permite ainda mesclar as matrizes treinadas aos pesos congelados [Hu et al. 2021]. O QLoRA mantém a base congelada em quatro bits enquanto os adaptadores são treinados e introduz o formato NormalFloat de quatro bits (NF4), a dupla quantização das constantes de quantização e otimizadores paginados [Dettmers et al. 2023]. Esses mecanismos geram, após o ajuste, ao menos duas formas de implantação: manter a base quantizada com o adaptador separado ou carregar a base em maior precisão e mesclar o adaptador.

O presente estudo se apoia nessas três linhas, mas difere de cada uma em pontos específicos. Ele usa o teste global e os pares do próprio PrimeVul, inclui uma heurística explícita de comprimento como referência de característica correlacionada ao rótulo e separa as categorias de decisão pareada da ordenação contínua dos escores. Também mede a memória e o tempo de duas implantações de cada ajuste 7B. Diferentemente de Risse e Böhme [2024], não aplica transformações de código. O TrapEval ajusta LLMs generativos com e sem raciocínio explícito; aqui, o ajuste usa uma cabeça de classificação, e a geração de texto aparece apenas na classificação por instrução YES/NO, sem exemplos e sem ajuste. Por isso, os números desses trabalhos não são comparados diretamente aos locais.

3. Materiais e métodos

3.1. Visão geral

O estudo trata a detecção como classificação binária de funções C/C++ e compara três grupos de configurações: variantes Instruct do Qwen2.5-Coder usadas por instrução textual, classificadores Base ajustados com LoRA e uma heurística de comprimento. O protocolo seguiu esta sequência. Primeiro, cada classificador foi treinado por até três épocas, e a época foi escolhida em dados de validação. Em seguida, dois limiares foram calibrados na validação global e aplicados ao teste sem recalibração: o de maior F1, às métricas globais e às categorias pareadas; o de FPR de até 0,5%, à taxa de falsos negativos e à de falsos positivos do teste global. Depois, os checkpoints 7B foram avaliados em duas implantações, com registro de memória e tempo. Por fim, a ordenação pareada do 7B foi resumida entre sementes, com intervalos por reamostragem e análises de sensibilidade. Este é um estudo retrospectivo e exploratório: as métricas foram recalculadas a partir das previsões registradas nas execuções originais, alinhadas às partições oficiais, e o conjunto de teste foi consultado enquanto as receitas eram desenvolvidas. Os procedimentos de alinhamento e a proveniência dos registros estão no Material Suplementar (Seções S1 e S2).

3.2. Conjunto de dados

Foi utilizada a distribuição original do PrimeVul, composta por funções C/C++ de projetos de código aberto, com rótulos derivados de commits de correção, deduplicação e divisão cronológica [Ding et al. 2024]. A Tabela 1 apresenta as partições usadas e os pares vulnerável/corrigida de cada uma.

Tabela 1. Partições do PrimeVul utilizadas: número de funções, funções rotuladas como vulneráveis e pares vulnerável/corrigida.

Partição	Funções	Vulneráveis	Pares
Treino	184.427	5.574	4.354
Validação	25.430	699	562
Teste	25.911	695	564

A classe vulnerável corresponde a 2,68% do teste global, o que torna a acurácia pouco informativa e motiva o uso de F1, precisão, revocação e taxas de erro. Cada par reúne uma função rotulada como vulnerável e outra rotulada como sua versão corrigida, no mesmo projeto e commit. Os 564 pares de teste pertencem a 550 grupos projeto/commit, unidade usada na reamostragem (Seção 3.6). Os classificadores e a heurística usaram contexto de 2.048 tokens, preservando o início da função; nos 465 pares de teste, as duas versões cabem integralmente nesse limite.

3.3. Configurações avaliadas

Nas avaliações por instrução, as variantes Qwen2.5-Coder-3B-Instruct e 7B-Instruct, carregadas em FP16, receberam uma instrução sem exemplos que pedia uma única palavra, YES ou NO, para cada função. A decodificação foi determinística, com até cinco novos tokens. Os registros históricos dessas rodadas indicam limite de 8.192 tokens para o

código. Um analisador aceitou a primeira palavra de decisão reconhecida; respostas indefinidas seriam contadas como não vulneráveis, mas nenhuma ocorreu nas quatro avaliações. Essas configurações produzem apenas decisões e, por isso, não entram na ordenação pareada.

A heurística de comprimento atribui a cada função um escore igual ao número de tokens visíveis após o corte em 2.048 tokens, obtido com o tokenizador do Qwen2.5-Coder. Sua função no estudo é servir de referência: ela não usa nenhuma informação sobre a vulnerabilidade, de modo que seu desempenho indica quanto uma característica superficial correlacionada ao rótulo consegue alcançar em cada métrica.

Os classificadores partiram dos modelos Base Qwen2.5-Coder-3B e 7B, com uma cabeça de duas saídas. O escore de vulnerabilidade é a diferença entre o logit da classe vulnerável e o da não vulnerável. No ajuste, a base foi carregada em NF4 com dupla quantização e computação em BF16. LoRA foi aplicada às projeções de atenção (q, k, v, o) e às projeções do bloco MLP (gate, up, down), com posto $r = 16$, $\alpha = 32$ e *dropout* de 0,05, e a cabeça de classificação também foi treinada. O otimizador foi o AdamW convencional, e não os otimizadores paginados do QLoRA, com taxa de aprendizado inicial de 2×10^{-4}, sem decaimento de pesos, aquecimento em 3% dos passos e decaimento linear. O lote efetivo teve 16 funções, com divisão por orçamento de tokens, *checkpointing* de gradientes e corte da norma do gradiente em 1,0. A perda foi a entropia cruzada, sem peso adicional para a classe vulnerável.

Três receitas de amostragem do treino foram exploradas no 3B. A primeira manteve todas as funções vulneráveis e sorteou igual número de não vulneráveis (1:1). A segunda usou o treino completo, na proporção original. A terceira formou o conjunto 1:1 escolhendo, sem reposição, para cada função vulnerável, a não vulnerável de comprimento tokenizado mais próximo, com o comprimento limitado a 2.048 tokens (1:1 com controle de tamanho). Essa última receita procura reduzir a diferença de tamanho entre as classes no treino, sem eliminar necessariamente outras características correlacionadas ao rótulo. O 7B usou somente essa receita, com três sementes de aleatoriedade (*seeds*); cada configuração 3B teve uma única semente. A Tabela 2 resume as configurações.

Tabela 2. Configurações de ajuste. Contagens de funções do treino usadas por receita; épocas executadas e época selecionada; horas de treino registradas. As horas não representam orçamento igual entre receitas. O custo financeiro das rodadas foi migrado para o Material Suplementar (Seção S5).

Configuração	Vulneráveis	Não vulneráveis	Épocas	Selecionada	Treino (h)
3B 1:1	5574	5574	3	1	3,37
3B proporção original	5574	178853	1	1	10,16
3B 1:1 + tamanho	5574	5574	3	3	4,98
7B 1:1 + tamanho, semente 1	5574	5574	3	3	7,89
7B 1:1 + tamanho, semente 2	5574	5574	3	2	7,89
7B 1:1 + tamanho, semente 3	5574	5574	3	3	7,91

Como as receitas diferem em composição, número de exemplos, épocas e duração, a comparação entre elas é descritiva. Ela não identifica efeito causal do balanceamento, do controle de tamanho ou da escala.

3.4. Seleção de época, implantação e recursos

Nas execuções originais, a época de cada classificador foi escolhida pela ordenação pareada estrita nos 562 pares de validação, contando empates como erro, e a AUC em uma amostra da validação global (699 vulneráveis e 2.796 não vulneráveis) serviu de critério de desempate. Depois, os empates passaram a receber meio crédito, como na métrica definida na Seção 3.5. Nos quatro treinos com controle de tamanho, os registros por época permitiram reconstruir retrospectivamente a seleção com esse critério, e a época escolhida não mudou. Nas receitas 3B 1:1 e de proporção original, os empates por época não foram registrados, e os checkpoints históricos foram mantidos (Material Suplementar, Seção S6); a configuração de proporção original teve uma única época.

Cada checkpoint 7B selecionado foi avaliado em duas implantações. Na implantação NF4, a base permaneceu quantizada e o adaptador separado; na implantação BF16, a base foi carregada em BF16 e o adaptador foi mesclado aos pesos. As duas compartilham semente e época, mas diferem em dois aspectos ao mesmo tempo: a representação da base e a incorporação do adaptador. Por isso, são tratadas como configurações completas de implantação, e não como um experimento que isola a quantização.

Os recursos foram registrados nas execuções originais em uma GPU NVIDIA A40, com PyTorch e Transformers (versões no Material Suplementar, Seção S11), lotes de até 32.768 tokens, incluindo preenchimento, e contexto de 2.048 tokens. A memória é a memória CUDA alocada, em GB decimais, medida após a carga do modelo. O pico foi reiniciado após a carga e a eventual mesclagem e observado durante a pontuação da validação, do teste e dos pares; por isso, pode não incluir o pico transitório da mesclagem. O tempo por função é o tempo de pontuação do teste dividido pelo número de funções, ou seja, um tempo amortizado em lotes, e não a latência de uma requisição individual. O custo estimado multiplica esse tempo pelo preço histórico de US$ 0,50 por hora de GPU e não corresponde a faturamento.

3.5. Regras de decisão e métricas

A decisão dos classificadores e da heurística segue a regra $s > t$, em que s é o escore e t o limiar. Dois limiares foram calibrados na validação global completa, cada um com uma finalidade. O primeiro maximiza F1 e serve às métricas globais e às categorias pareadas. O segundo é o menor limiar, entre os escores das n funções não vulneráveis da validação, que pela regra $s > t$ admite no máximo $\lfloor 0{,}005\, n \rfloor$ falsos positivos, o que mantém a taxa de falsos positivos (FPR) em no máximo 0,5% na validação; ele aproxima o cenário de poucos falsos alarmes do VD-Score e serve para medir a taxa de falsos negativos (FNR) no teste global. Ambos foram aplicados ao teste sem recalibração. Como o segundo limiar foi escolhido na validação, a FPR observada no teste pode ultrapassar 0,5% e é reportada junto à FNR. O procedimento separa calibração e teste e usa desigualdade estrita, ao passo que a implementação oficial do VD-Score escolhe o ponto na própria curva ROC do conjunto avaliado; por isso, os valores locais não reproduzem o VD-Score oficial.

No teste global, foram calculados F1, precisão, revocação, FPR e AUC ROC; a AUC só se aplica às configurações com escore. No teste pareado, cada par recebe uma de quatro categorias, segundo as decisões sobre a versão vulnerável e a corrigida, nessa ordem: P-C, (1,0), ambas corretas; P-V, (1,1), ambas vulneráveis; P-B, (0,0), ambas não vulneráveis; e P-R, (0,1), decisões invertidas. As categorias dependem do limiar. A

ordenação pareada, ao contrário, independe de limiar e resume a frequência com que a versão vulnerável recebe escore maior que a corrigida, dando meio crédito aos empates:

$$A_{\text{par}} = \frac{1}{N} \sum_{j=1}^{N} \left[\mathbf{1}(s_{v,j} > s_{c,j}) + \tfrac{1}{2}\, \mathbf{1}(s_{v,j} = s_{c,j}) \right]$$

em que $s_{v,j}$ e $s_{c,j}$ são os escores da versão vulnerável e da corrigida no par j, e $N = 564$. Por causa do meio crédito, A_{par} não é a fração de pares em que a versão vulnerável recebeu escore maior, e um escore sem relação com as versões tende a 50%. Valores abaixo de 50% indicam que, entre os pares sem empate, a versão corrigida recebeu escore maior com mais frequência que a vulnerável. Ordenar bem os pares não implica entendimento do código, e uma ordenação favorável não garante que o limiar produza a categoria P-C.

3.6. Análise estatística e sensibilidades

No 7B, a variação entre treinamentos foi descrita pelos valores de cada semente, pela média e pelo desvio padrão amostral das três execuções; as configurações 3B não foram agregadas como repetições. A incerteza amostral da ordenação no teste foi estimada por *bootstrap* percentil de 95% com 10.000 réplicas. A unidade reamostrada foi o grupo projeto/commit: os 550 grupos foram sorteados com reposição, mantendo todos os pares de cada grupo, como no *bootstrap* de agrupamentos discutido por Field e Welsh [2007]. As propriedades desse procedimento dependem do modelo de dependência considerado, e a dependência entre commits do mesmo projeto não foi eliminada. Em cada réplica, os mesmos grupos foram usados para todas as configurações e sementes, o que permite calcular a diferença NF4 - BF16 de forma pareada. Os intervalos são condicionais aos três modelos treinados e não descrevem a variação de treinamentos futuros. As três sementes também não triplicam o tamanho da amostra para 1.692 pares. Não foram realizados testes de hipótese.

Duas sensibilidades retrospectivas acompanham a análise principal. A primeira restringe a ordenação aos 465 pares sem truncamento. A segunda remove cinco pares cuja correspondência não representa a correção da mesma função (Seção 4.4); sua interseção com o recorte sem truncamento contém 460 pares. Ambas alteram a composição do teste e não substituem os 564 pares oficiais como referência.

4. Resultados

4.1. Classificação por instrução

A Tabela 3 mostra como as variantes Instruct se comportaram com a instrução YES/NO, tanto no teste global quanto nos pares.

Tabela 3. Classificação por instrução no teste global (25.911 funções) e categorias no teste pareado (564 pares), pela decisão YES/NO. Contexto do código registrado: 8.192 tokens.

Modelo	F1	Precisão	Revocação	FPR (%)	P-C	P-V	P-B	P-R
3B Instruct	0,119	0,102	0,144	3,49	16	67	463	18
7B Instruct	0,062	0,038	0,160	11,00	14	78	454	18

O F1 foi baixo nas duas escalas, e o 7B ficou abaixo do 3B. A revocação um pouco maior do 7B, 0,160 contra 0,144, veio acompanhada de FPR de 11,00%, contra 3,49%, o que reduziu sua precisão a 0,038. Nos pares, a categoria P-B, em que as duas versões são declaradas não vulneráveis, reuniu 463 e 454 dos 564 casos, enquanto P-C ocorreu em 16 e 14 pares. Esses resultados descrevem a instrução e a regra de decisão utilizadas, e não todo o espaço de instruções possíveis.

4.2. Desempenho global e ordenação pareada

A Tabela 4 apresenta o desempenho global das configurações com escore, e a Tabela 5 mostra, para as mesmas configurações, as categorias pareadas e a ordenação. As duas tabelas devem ser lidas em conjunto, porque o contraste entre elas responde à primeira questão do estudo.

Tabela 4. Teste global, 25.911 funções. F1, precisão, revocação e FPR usam o limiar de maior F1 na validação. FNR* e FPR* usam outro limiar, calibrado para FPR $\leq$ 0,5% na validação; FPR* é o valor observado no teste. s1–s3 indicam as sementes do 7B.

Configuração	F1	Precisão	Revocação	FPR (%)	AUC	FNR* (%)	FPR* (%)
Heurística de comprimento	0,200	0,152	0,294	4,53	0,810	100,00	0,00
3B 1:1	0,195	0,142	0,312	5,19	0,795	94,10	0,54
3B proporção original	0,262	0,200	0,380	4,19	0,851	88,06	0,65
3B 1:1 + tamanho	0,121	0,083	0,224	6,86	0,746	96,26	0,63
7B NF4 s1	0,114	0,077	0,222	7,36	0,738	97,99	0,69
7B BF16 s1	0,119	0,081	0,229	7,19	0,740	98,13	0,67
7B NF4 s2	0,142	0,103	0,229	5,47	0,726	98,71	0,38
7B BF16 s2	0,148	0,115	0,207	4,41	0,728	98,71	0,38
7B NF4 s3	0,127	0,084	0,258	7,75	0,765	98,27	0,56
7B BF16 s3	0,124	0,087	0,220	6,39	0,765	98,27	0,52

Tabela 5. Teste pareado oficial, 564 pares. As categorias usam o limiar de maior F1 na validação. Empates são pares com escores iguais nas duas versões. A ordenação dá meio crédito aos empates e independe de limiar; a coluna 465 considera apenas pares com as duas versões não truncadas.

Configuração	P-C	P-V	P-B	P-R	Empates	Ord. 564 (%)	Ord. 465 (%)
Heurística de comprimento	2	156	398	8	124	21,10	15,16
3B 1:1	13	155	372	24	52	42,38	39,03
3B proporção original	6	200	344	14	43	37,50	34,09
3B 1:1 + tamanho	6	120	422	16	37	49,20	48,71
7B NF4 s1	7	116	435	6	39	55,23	54,09
7B BF16 s1	6	125	425	8	39	53,28	51,94
7B NF4 s2	5	128	423	8	40	45,04	42,37
7B BF16 s2	7	113	434	10	34	44,68	41,94
7B NF4 s3	6	134	412	12	32	51,60	51,61
7B BF16 s3	8	113	434	9	32	51,95	52,04

O contraste mais nítido envolve a heurística de comprimento. Usando apenas o número de tokens, ela alcançou AUC global de 0,810 e F1 de 0,200, valores próximos aos do classificador 3B 1:1. A AUC indica que, no teste global, as funções vulneráveis

tendem a ser mais longas que as não vulneráveis. Nos pares, porém, a ordenação pareada da mesma heurística foi de apenas 21,10%. Como esse valor está abaixo de 50%, a versão corrigida teve mais tokens visíveis que a vulnerável com mais frequência do que o inverso, entre os pares sem empate. Os 124 empates são pares em que as duas versões têm o mesmo número de tokens visíveis; entre eles estão necessariamente os pares cujas duas versões atingem o teto de 2.048 tokens e os 32 pares com entradas idênticas após o corte. Assim, a característica que separa as classes no teste global não separa, e tende a inverter, as duas versões de uma mesma correção.

Nas configurações 3B, os classificadores repetem esse contraste em menor grau. As duas de maior F1 e AUC na Tabela 4, a de proporção original (0,262 e 0,851) e a 1:1 (0,195 e 0,795), tiveram ordenações pareadas de 37,50% e 42,38%, abaixo das obtidas pelas configurações com controle de tamanho, que ficaram entre 44,68% e 55,23% com F1 entre 0,114 e 0,148 e AUC entre 0,726 e 0,765. Esse contraste é compatível com a hipótese de que parte do desempenho global dependa de características correlacionadas ao rótulo, como o comprimento, e não da diferença introduzida pela correção. O desenho, contudo, não identifica essa causa, porque as receitas também diferem em composição, número de exemplos e duração. Dentro do 7B, que usou uma única receita, os seis casos observados não formam um padrão descritivo uniforme entre métricas globais e ordenação: a semente 2 teve o maior F1, a menor AUC e as menores ordenações, a semente 3 teve a maior AUC, e a semente 1, as maiores ordenações. Essa observação descreve os casos da Tabela 4 e da Tabela 5 e não estima uma associação estatística.

As categorias pareadas complementam essa leitura. Com o limiar de maior F1, a categoria P-B predominou em todas as configurações, e P-C ficou entre 2 e 13 pares; nas seis avaliações 7B, entre 5 e 8. Mesmo quando a ordenação se aproxima ou supera 50%, a diferença entre os escores das duas versões raramente atravessa o limiar. Uma ordem relativa favorável, portanto, não se converte em decisões corretas para ambas as versões.

O segundo limiar mostra outra face das mesmas configurações. Quando a FPR é limitada a 0,5% na validação, a FNR no teste foi de 88,06% na configuração 3B de proporção original e ficou entre 97,99% e 98,71% no 7B. Apenas as avaliações da semente 2 do 7B e a heurística mantiveram a FPR do teste em até 0,5%; nas demais, a FPR observada ficou entre 0,52% e 0,69%. Na heurística, esse limiar coincidiu com o teto de 2.048 tokens, e pela regra estrita nenhuma função o ultrapassou, o que resultou em FNR de 100% e FPR nula. Esse resultado decorre da combinação entre truncamento e regra de decisão e não contradiz a AUC de 0,810. Em um cenário de poucos falsos alarmes, nenhuma configuração deixou de perder a maior parte das funções vulneráveis.

4.3. Dispersão entre sementes e incerteza no 7B

A Tabela 6 resume a ordenação das três sementes 7B em cada implantação, separando a dispersão entre treinamentos da incerteza amostral do teste.

As médias foram 50,62% em NF4 e 49,97% em BF16, com desvios padrão de 5,17 e 4,63 pontos percentuais entre sementes. Esses desvios são da mesma ordem que a amplitude dos intervalos condicionais das médias, e as sementes isoladas apontam em direções opostas: a semente 1 ficou acima de 50% nas duas implantações, e a semente 2, abaixo. Uma única semente poderia, assim, sugerir conclusões contrárias. A diferença média NF4 - BF16 foi de 0,65 ponto percentual, com intervalo de -1,22 a 2,54. Esse

Tabela 6. Ordenação pareada do 7B: média das três sementes, desvio padrão entre sementes e intervalo percentil de 95% por *bootstrap* de 550 grupos projeto/commit (10.000 réplicas), condicional aos três modelos observados. O intervalo da diferença mantém os mesmos grupos nas duas implantações.

Medida	Estimativa (%) ou p.p.	DP sementes (p.p.)	IC95% inferior	IC95% superior
NF4	50,62	5,17	47,92	53,33
BF16	49,97	4,63	47,14	52,73
NF4 - BF16 (p.p.)	0,65	—	-1,22	2,54

intervalo inclui diferenças nas duas direções e não distingue as implantações; ele também não estabelece equivalência, que exigiria uma margem definida antes da análise. Já os intervalos das médias, de 47,92% a 53,33% em NF4 e de 47,14% a 52,73% em BF16, incluem 50%, o que não prova que os modelos ordenem ao acaso.

4.4. Sensibilidades ao truncamento e à correspondência dos pares

A última coluna da Tabela 5 repete a ordenação nos 465 pares sem truncamento. A heurística caiu de 21,10% para 15,16%. No 7B, as maiores reduções ocorreram na semente 2, de 45,04% para 42,37% em NF4 e de 44,68% para 41,94% em BF16, enquanto a semente 3 praticamente não mudou. Após o corte em 2.048 tokens, 32 pares têm sequências de entrada idênticas nas duas versões, ao passo que as avaliações 7B registraram de 32 a 40 empates; parte dos empates decorre, portanto, de entradas idênticas, mas um empate de escore não prova essa identidade. Como o recorte também muda a composição do teste, a diferença não pode ser atribuída apenas ao truncamento.

Uma inspeção focalizada examinou os 15 pares de teste cujas versões diferem no cabeçalho ou na assinatura. A verificação nos repositórios de origem (Material Suplementar, Seção S3) confirmou cinco casos em que a função rotulada como corrigida já existia antes do commit e corresponde a outra função, e não à versão corrigida da vulnerável. Removidos esses cinco pares, a maior alteração absoluta nas seis avaliações 7B foi de 0,22 ponto percentual, e o padrão descrito nas Seção 4.2 e 4.3 se manteve (Tabela S1 do Material Suplementar). Esse resultado indica que esses casos não alteram as conclusões, mas não certifica os 559 pares restantes nem estima a frequência desse problema no PrimeVul.

4.5. Recursos das implantações 7B

A Tabela 7 apresenta os recursos registrados para cada checkpoint 7B nas duas implantações.

A implantação NF4 alocou 4,63 GB após a carga, cerca de 67% menos que os 14,15 GB da BF16, e atingiu pico de 12,49 GB, contra 20,76 GB, uma redução de cerca de 40%. Em contrapartida, o tempo amortizado foi maior em NF4, de 59,6 a 60,0 ms por função, contra 46,8 a 47,1 ms em BF16. Com o mesmo preço por hora, o custo estimado acompanhou essa diferença. Nesse ambiente, portanto, a implantação NF4 trocou memória por tempo, sem diferença distinguível na ordenação pareada (Tabela 6).

Tabela 7. Recursos do 7B na NVIDIA A40, com Transformers, lotes de até 32.768 tokens e contexto de 2.048 tokens. Memória CUDA alocada após a carga e pico na janela de pontuação da validação, do teste e dos pares, em GB decimais. Tempo amortizado por função no teste, em lotes. Custo estimado da pontuação do teste ao preço histórico de US$ 0,50/h. As sementes não são repetições independentes da medição de desempenho computacional.

Configuração	Após carga (GB)	Pico (GB)	Tempo/função (ms)	Custo estimado do teste (US$)
7B NF4 s1	4,63	12,49	59,6	0,2145
7B BF16 s1	14,15	20,76	46,8	0,1684
7B NF4 s2	4,63	12,49	59,8	0,2152
7B BF16 s2	14,15	20,76	46,9	0,1687
7B NF4 s3	4,63	12,49	60,0	0,2160
7B BF16 s3	14,15	20,76	47,1	0,1696

5. Discussão

O principal resultado é a dissociação entre desempenho global e ordenação pareada. Uma heurística sem qualquer informação sobre vulnerabilidades obteve AUC global de 0,810 e, entre os pares não empatados, atribuiu escore maior à versão corrigida na maioria dos casos. O classificador com maior F1 também teve ordenação pareada abaixo de 50%, e as configurações treinadas com controle de tamanho ficaram em torno de 50% com desempenho global menor. Esse quadro é coerente com a observação de Chakraborty et al. [2020] de que detectores profundos frequentemente não aprendem características da vulnerabilidade e com o achado de Risse e Böhme [2024] de que modelos treinados em conjuntos típicos não distinguem funções vulneráveis de suas correções. Os protocolos, porém, diferem: esses últimos autores usaram transformações que preservam a semântica e um conjunto próprio de pares, enquanto este estudo usa os pares do PrimeVul e uma heurística explícita, sem transformações. A contribuição local está em tornar visível uma característica concreta, o comprimento, que produz discriminação global e ordenação pareada desfavorável no mesmo conjunto. A semelhança com o TrapEval, em que escores altos no contraste com código não relacionado conviveram com falhas nas versões corrigidas [Huang et al. 2026], é apenas qualitativa: o teste global do PrimeVul não é o conjunto V2N, e os modelos e as formas de ajuste diferem.

A calibração faz parte do resultado, e não apenas de sua apresentação. Com o limiar de maior F1, as categorias pareadas concentraram-se em P-B; com o limiar de baixa FPR, a FNR passou de 88% em todas as configurações, e a restrição escolhida na validação nem sempre se manteve no teste. A avaliação com restrição de falsos alarmes proposta por Ding et al. [2024] torna visível essa limitação, que o F1 sozinho não mostra. A leitura da AUC também exige cautela: em dados desbalanceados, a curva ROC pode ser enganosa quanto à confiabilidade do classificador, e a precisão fornece informação que ela não mostra [Saito e Rehmsmeier 2015]. No caso local, a heurística combinou AUC de 0,810 com precisão de 0,152 no limiar de maior F1. Como a análise não calculou a área sob a curva precisão-revocação, essa observação se limita aos pontos de operação reportados.

O desempenho fraco da classificação por instrução é coerente com a literatura de avaliação de LLMs, mas não se explica por ela. Ullah et al. [2024] relataram respostas não

determinísticas, raciocínio infiel e baixa robustez em oito LLMs. Este estudo usou uma única instrução sem exemplos, com decodificação determinística, e observou que a variante maior teve mais falsos positivos e menor F1. Esse resultado se limita a essa instrução: outras formulações, exemplos ou raciocínio explícito poderiam alterar o comportamento, e a família Qwen2.5-Coder não integrou o grupo avaliado por esses autores.

Os cinco pares cuja versão corrigida corresponde a outra função mostram que a qualidade da avaliação pareada depende da construção de cada par. Chen et al. [2023] quantificaram ruído de rótulo em conjuntos derivados de commits de correção, e Ding et al. [2024] apontaram problemas de rotulação e duplicação que motivaram o PrimeVul. A inspeção local foi focalizada e não estima uma taxa, mas indica que verificar a correspondência entre as versões é um passo necessário antes de interpretar ordenações próximas de 50%. A sensibilidade de 0,22 ponto percentual indica que esses casos não mudam o quadro do 7B, sem garantir que o restante dos pares esteja correto.

Por fim, a comparação entre implantações tem uma leitura prática delimitada. A mesclagem permitida pela LoRA [Hu et al. 2021] e a base em NF4 do QLoRA [Dettmers et al. 2023] levaram, neste ambiente, a uma troca entre cerca de dois terços menos memória alocada após a carga e maior tempo amortizado por função. Como a representação da base e a incorporação do adaptador mudaram juntas, esse resultado orienta a escolha entre as duas configurações completas avaliadas, mas não indica o efeito isolado da quantização nem a configuração mais econômica em outros equipamentos ou cargas.

6. Limitações

Quanto à validade interna, o conjunto de teste foi consultado durante o desenvolvimento das receitas, e a análise recalculou métricas de previsões já existentes. Os limiares foram escolhidos na validação, mas isso não elimina o contato prévio com o teste, e os resultados não constituem avaliação confirmatória independente. A época foi escolhida nas execuções originais pela ordenação pareada estrita, com a AUC como desempate; apenas a verificação com meio crédito para empates foi retrospectiva. Essa verificação manteve a época nos quatro treinos com controle de tamanho, mas não pôde ser feita nas receitas 3B 1:1 e de proporção original, porque os empates por época não foram registrados; na segunda, que teve uma única época, não havia alternativa a comparar. As receitas diferem em composição, número de exemplos, épocas e duração, e a comparação NF4/BF16 altera ao mesmo tempo a representação da base e a mesclagem do adaptador. Nenhuma dessas comparações identifica efeitos causais.

Quanto à validade estatística, cada configuração 3B tem uma única semente, e o 7B tem três. O desvio padrão de cerca de 5 pontos percentuais mostra que essa quantidade é pequena para caracterizar a variação entre treinamentos. Os intervalos por *bootstrap* são condicionais aos modelos observados e supõem grupos projeto/commit intercambiáveis, embora commits do mesmo projeto possam ser dependentes; o procedimento de reamostragem por agrupamentos não garante, por si, a cobertura nominal nessas condições. A ausência de diferença distinguível entre NF4 e BF16 não equivale a equivalência, e nenhum teste de hipótese foi realizado.

Quanto à validade de construção, cinco pares oficiais não representam a correção da mesma função, e os demais 559 não foram certificados; a inspeção cobriu apenas os pares com cabeçalhos divergentes. O corte em 2.048 tokens remove parte do código das

funções longas e torna idênticas as entradas de 32 pares. A ordenação pareada dá meio crédito aos empates e, por isso, não equivale à fração de pares ordenados a favor da versão vulnerável. Ordenar a versão vulnerável acima da corrigida não demonstra entendimento da vulnerabilidade, e uma ordenação próxima de 50% não demonstra ausência dele. A classificação por instrução usou contexto e variantes diferentes dos classificadores. Os escores registrados com cinco casas decimais limitam a recuperação de empates e limiares (Material Suplementar, Seção S7). A divisão cronológica, a deduplicação e os rótulos do PrimeVul não provam ausência de contaminação no pré-treinamento dos modelos.

Quanto à validade externa, os resultados se referem a uma família de modelos, um conjunto de dados, uma GPU e uma biblioteca de inferência. A memória e o tempo foram registrados uma vez por checkpoint, o pico pode não incluir a mesclagem, e o tempo amortizado em lotes não corresponde à latência individual. Energia, faturamento real e outros métodos de quantização não foram medidos.

7. Conclusão

Este trabalho analisou configurações Qwen2.5-Coder de 3B e 7B no PrimeVul, combinando o teste global, os 564 pares vulnerável/corrigida e os recursos de duas implantações. Foram avaliados modelos por instrução, classificadores ajustados com LoRA sobre base NF4 e uma heurística de comprimento, com limiares calibrados na validação e ordenação pareada independente de limiar.

A evidência sustenta três observações para as configurações executadas. Primeiro, desempenho global e discriminação pareada não andaram juntos: a heurística de comprimento alcançou AUC de 0,810 com ordenação pareada de 21,10%, e o classificador de maior F1 (0,262) teve ordenação pareada de 37,50%, enquanto as configurações com controle de tamanho ficaram em torno de 50% com desempenho global menor; dentro do 7B, esse contraste não se repetiu de forma uniforme. Segundo, a variação entre as três sementes do 7B, com médias de 50,62% em NF4 e 49,97% em BF16 e desvio padrão de cerca de 5 pontos percentuais, foi grande em relação às diferenças entre implantações. Terceiro, a implantação NF4 com adaptador separado usou menos memória alocada e mais tempo amortizado que a BF16 com adaptador mesclado, sem diferença distinguível na ordenação. Nessas configurações e nesse conjunto, portanto, as métricas globais não bastaram para descrever a discriminação entre as versões de uma correção, e a leitura conjunta com a avaliação pareada, a calibração e as condições de implantação alterou a interpretação dos resultados.

A continuidade deste estudo requer uma avaliação confirmatória com receitas, critérios e limiares definidos antes do acesso ao teste, mais sementes por configuração e orçamento de treino comparável entre receitas. Também requer a revisão sistemática da correspondência dos pares e uma comparação que separe a quantização da mesclagem, por exemplo com BF16 e adaptador separado. Medições repetidas de memória, tempo e latência individual em mais de um ambiente permitiriam avaliar se a troca observada se mantém.

Referências

Chakraborty, S.; Krishna, R.; Ding, Y.; Ray, B. (2020). Deep Learning based Vulnerability Detection: Are We There Yet?. arXiv:2009.07235. DOI: 10.48550/arXiv.2009.07235.

Chen, Y.; Ding, Z.; Alowain, L.; Chen, X.; Wagner, D. (2023). DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection. 26th International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2023). DOI: 10.1145/3607199.3607242.

Dettmers, T.; Pagnoni, A.; Holtzman, A.; Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. arXiv:2305.14314v1. DOI: 10.48550/arXiv.2305.14314.

Ding, Y.; Fu, Y.; Ibrahim, O.; Sitawarin, C.; Chen, X.; Alomair, B.; Wagner, D.; Ray, B.; Chen, Y. (2024). Vulnerability Detection with Code Language Models: How Far Are We?. arXiv:2403.18624v2. DOI: 10.48550/arXiv.2403.18624.

Field, C. A.; Welsh, A. H. (2007). Bootstrapping clustered data. Journal of the Royal Statistical Society: Series B (Statistical Methodology), 69(3), 369–390. DOI: 10.1111/j.1467-9868.2007.00593.x.

Fu, M.; Tantithamthavorn, C. (2022). LineVul: A Transformer-based Line-Level Vulnerability Prediction. 19th International Conference on Mining Software Repositories (MSR 2022), 608–620. DOI: 10.1145/3524842.3528452.

Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685v2. DOI: 10.48550/arXiv.2106.09685.

Huang, F.; Sun, Y.; Zhang, F.; Yang, Z.; Liu, H.; Liu, Y. (2026). Do Fine-Tuned LLMs Understand Vulnerabilities? An Investigation into the Semantic Trap. arXiv:2601.22655v3. DOI: 10.48550/arXiv.2601.22655.

Hui, B.; Yang, J.; Cui, Z.; Yang, J.; Liu, D.; Zhang, L.; Liu, T.; Zhang, J.; Yu, B.; Lu, K.; Dang, K.; Fan, Y.; Zhang, Y.; Yang, A.; Men, R.; Huang, F.; Zheng, B.; Miao, Y.; Quan, S.; Feng, Y.; Ren, X.; Ren, X.; Zhou, J.; Lin, J. (2024). Qwen2.5-Coder Technical Report. arXiv:2409.12186v3. DOI: 10.48550/arXiv.2409.12186.

Risse, N.; Böhme, M. (2024). Uncovering the Limits of Machine Learning for Automatic Vulnerability Detection. 33rd USENIX Security Symposium (USENIX Security 24), 4247–4264.

Saito, T.; Rehmsmeier, M. (2015). The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. PLOS ONE, 10(3), e0118432. DOI: 10.1371/journal.pone.0118432.

Ullah, S.; Han, M.; Pujar, S.; Pearce, H.; Coskun, A.; Stringhini, G. (2024). LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?): A Comprehensive Evaluation, Framework, and Benchmarks. arXiv:2312.12575v3. DOI: 10.48550/arXiv.2312.12575.

Zhou, Y.; Liu, S.; Siow, J.; Du, X.; Liu, Y. (2019). Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks. Advances in Neural Information Processing Systems, 32.